

Fast PUF-Based Four-Phase Authentication for UAV Swarms: An OMNeT++ Evaluation with SHA3-256 and SPONGENT-160 Hash Modes

Md Ziyad Hussain

Dept. of Computer Science and Engineering
Indian Institute of Information Technology Guwahati
Bongora, Assam 781015, India
ziyad.hussain23b@iiitg.ac.in

Rakesh Matam

Dept. of Computer Science and Engineering
Indian Institute of Information Technology Guwahati
Bongora, Assam 781015, India
rakesh@iiitg.ac.in

Abstract—Unmanned Aerial Vehicle (UAV) swarms require fast, lightweight, and capture-resilient authentication for real-time coordination. Public-key alternatives (RSA-2048, ECC-256) add millisecond-scale handshake overhead and rely on private keys stored in non-volatile memory—keys that are exposed when a UAV is physically captured. We present a four-phase Physical Unclonable Function (PUF)-based authentication protocol covering: enrollment with a BCH(255,131,18) fuzzy extractor; UAV–Ground Station mutual authentication; direct full-mesh UAV–UAV peer authentication via a swarm-wide credential issued in Phase 2; and symmetric session-key establishment with optional Curve25519 forward secrecy. The protocol uses only hash, XOR, and PUF primitives, and is fully implemented in OMNeT++ 6.3 for a ten-UAV stadium-surveillance scenario (500×300 m). We characterize two hash primitives back-to-back: SHA3-256 truncated to 160 bits (via OpenSSL EVP) and a SPONGENT-160-style lightweight sponge. Measured end-to-end Phase 2 latency is 0.699 ms (SHA3) and 14.044 ms (SPONGENT); Phase 3 per-pair latency is 0.476 ms and 14.535 ms; communication overhead is 175 B and 104 B per phase, with 100% authentication success across all ten UAV–GS exchanges and 45 peer pairs. Per-message timing breakdowns localize cost to PUF evaluation, BCH decoding, and hash operations, enabling a synthesis-based projection in which the SHA3–SPONGENT gap collapses from 20–30× in software to 2–5× on FPGA/ASIC. Our SHA3 mode is 11–21× faster than RSA/ECC baselines while removing the stored-key attack surface entirely.

Index Terms—UAV swarm authentication, Physical Unclonable Function, PUF, SPONGENT, SHA3, lightweight cryptography, fuzzy extractor, BCH error correction, OMNeT++, peer-to-peer authentication.

I. INTRODUCTION

UNMANNED Aerial Vehicles (UAVs) are increasingly deployed as cooperating swarms for stadium surveillance, border patrol, search-and-rescue, agricultural sensing, and disaster response [1]. A common operational pattern—ten or more drones cooperatively covering a fixed perimeter—demands sub-millisecond mutual authentication between each UAV and a Ground Station (GS) and, increasingly, between UAV pairs without GS mediation when relay latency is unacceptable or the GS link is intermittent.

Conventional authentication options are poorly suited to this regime. **RSA-2048 PKI** requires 8–15 ms per signature

verification on an ARM Cortex-A72 [2], growing to nearly a second for full-mesh authentication of a ten-UAV swarm and adding 256–512 B certificates per exchange. **ECC-256** [3] halves the latency but still depends on private keys held in non-volatile memory—recoverable from a captured UAV by cold-boot or side-channel attacks. **Pre-shared keys (PSK)** reach sub-millisecond latency but compromise the entire swarm if any device is captured, and require $O(N^2)$ pairwise keys.

Physical Unclonable Functions (PUFs) provide a hardware root of trust without any stored key: the response to a challenge is produced by the silicon’s manufacturing variations, is unique to each device, and cannot be cloned even by the manufacturer [6]. A PUF-based protocol shifts the attack surface from extractable memory to physical capture-and-clone attacks that today’s adversaries cannot mount. The challenge is that PUF responses are noisy (3–10% bit-flip rate); a BCH-based fuzzy extractor [11], [12] recovers the enrollment-time response despite environmental drift.

Contributions

- 1) A complete four-phase PUF-based authentication protocol covering enrollment, UAV–GS authentication, GS-less full-mesh UAV–UAV peer authentication, and session-key derivation. Phase 3 avoids GS round-trips by using a swarm-wide credential issued in Phase 2 as the shared root of trust.
- 2) An OMNeT++ 6.3 reference implementation with a composite link-delay model, a 128-stage Arbiter PUF simulator, a from-scratch BCH(255,131,18) fuzzy extractor with Berlekamp-Massey decoding, and a dual-mode hash wrapper with per-mode timing accumulators.
- 3) End-to-end performance characterization across 10 UAV–GS authentications and 45 peer pairs, with per-message latency decomposition into UAV-side, GS-side, and network components.
- 4) A synthesis-based software-vs.-hardware projection showing that SPONGENT’s software cost becomes negligible on FPGA/ASIC, bringing total ten-UAV swarm authentication to ~ 29 ms regardless of hash mode.

The remainder of the paper covers related work (§II), protocol (§III), implementation (§IV), evaluation (§V), security and discussion (§VI), and conclusions (§VII).

II. RELATED WORK

PUF-based UAV authentication. Gope and Sikdar [2] proposed a hash/XOR-only PUF authentication for UAV networks, reporting 10–50 ms latency on a Raspberry Pi 4B, but addressing only UAV-to-GS authentication. Li *et al.* [4] extended PUF authentication to cross-domain emergency-rescue UAVs at the cost of 640 B overhead and 45–80 ms latency on embedded ARM. Khan *et al.* [5] combined PUF with biometrics for 6G UAV networks; their formal real-or-random (ROR) model security analysis provides a strong verification framework. Dey and Hossain [16] proposed MITM- and DoS-resistant PUF authentication via sensor-initiated registration. Xu *et al.* [7] introduced dynamic CRP rotation to prevent database exhaustion, which we adopt.

Peer-to-peer authentication. Direct UAV–UAV authentication remains under-explored. Bera *et al.* [3] achieved sub-millisecond P2P authentication using pre-shared symmetric keys, but lacked a PUF hardware root of trust. Banerjee *et al.* [18] identified vulnerabilities in prior UAV–GS authentication schemes (impersonation, session-key compromise, lack of forward secrecy), motivating our explicit security analysis. To our knowledge, no prior work characterizes a full-mesh PUF-based peer authentication scheme end-to-end in a discrete-event network simulator with per-message timing.

Lightweight cryptography. Bogdanov *et al.* [8] introduced SPONGENT, achieving 2,190 gate equivalents (GE) for SPONGENT-160; the bit-level permutation requires zero logic (only wire routing). SHA3/Keccak [9], [21] is software-fast ($\sim 0.4 \mu\text{s}$ per call on modern x86) but requires $\sim 38,000$ GE in hardware. The crossover between these two regimes motivates our dual-mode evaluation.

III. PROPOSED PROTOCOL

A. System and Threat Model

A swarm consists of one trusted GS and N UAVs. Each UAV embeds a 128-stage Arbiter PUF, a lightweight processor, and a wireless link. We adopt the Dolev-Yao adversarial model [14] augmented with physical capture: the adversary may extract NVM but cannot clone PUF hardware (tamper-evident probing destroys the response). Hashes are pre-image and collision resistant; PUFs are unclonable and unpredictable.

Notation. TID_i is the temporary identity of UAV i , $C_i^{(k)}$ the k -th challenge, $R_i^{(k)} = \text{PUF}_i(C_i^{(k)})$ the noisy response, $H_i^{(k)}$ its BCH helper data, $h(\cdot)$ a 160-bit hash, $\oplus$ bitwise XOR, $\parallel$ concatenation, T_x a 32-bit timestamp, N_x, n_x random 128-bit nonces.

B. Phase 1: Secure Enrollment

Enrollment is performed once per UAV in a physically secure facility. For each UAV i and each of $n \geq 12$ random

Algorithm 1 Phase 2 UAV–GS Mutual Authentication

- 1: **UAV** $\rightarrow$ **GS**: $\langle \text{TID}_i, T_1, N_1, h(\text{TID}_i \parallel T_1 \parallel N_1) \rangle$
- 2: **GS**: verify hash, look up TID, fetch $(C_i^{(k)}, H_i^{(k)}, R_i^{(k)})$, draw N_2
- 3: **GS** $\rightarrow$ **UAV**: $\langle C_i^{(k)}, N_2, h(\text{TID}_i \parallel C_i^{(k)} \parallel N_2 \parallel T_1) \rangle$
- 4: **UAV**: verify GS hash; $R^n \leftarrow \text{PUF}_i(C_i^{(k)})$
- 5: **UAV**: $R^m \leftarrow R^n \oplus h(N_2)$; $\text{SK}_i \leftarrow h(R^n \parallel N_1 \parallel N_2 \parallel T_1)$
- 6: **UAV** $\rightarrow$ **GS**: $\langle R^m, h(R^m \parallel \text{SK}_i) \rangle$
- 7: **GS**: recover R^n , compute $\text{ecc} = H_i^{(k)} \oplus R^n$, decode $R \leftarrow \text{BCH_decode}(R^n, \text{ecc})$
- 8: **GS**: check $\text{HD}(R, R_i^{(k)}) \leq \tau$, verify token, advance CRP index
- 9: **GS** $\rightarrow$ **UAV**: $\langle \text{success}, \text{Cred} \rangle$

challenges, the GS records $(C_i^{(k)}, R_i^{(k)})$ and computes helper data following the fuzzy-extractor Gen construction:

$$H_i^{(k)} = \text{BCH_encode}(R_i^{(k)}) \oplus R_i^{(k)}. \quad (1)$$

Under Maes’ helper-data-masking analysis [12], H is computationally independent of R . After enrollment the UAV holds only its TID; no key material of any kind is stored.

C. Phase 2: UAV–GS Mutual Authentication

Phase 2 is a four-message protocol (Alg. 1) executed when a UAV joins the swarm. The UAV first authenticates itself with a hashed nonce; the GS issues a fresh CRP challenge and its own authenticator; the UAV evaluates its PUF, masks the response, and returns an authentication token; the GS performs BCH correction, verifies the token, and on success issues a swarm-wide credential Cred .

D. Phase 3: UAV–UAV Peer Authentication

After Phase 2, each pair (i, j) with $i < j$ runs a three-message exchange using Cred as the shared root of trust:

$$P_1 : \text{UAV}_i \rightarrow \text{UAV}_j : \langle n_i, h(i \parallel j \parallel n_i \parallel \text{Cred}) \rangle, \quad (2)$$

$$P_2 : \text{UAV}_j \rightarrow \text{UAV}_i : \langle n_j, h(j \parallel i \parallel n_j \parallel \text{Cred}) \rangle, \quad (3)$$

$$P_3 : \text{UAV}_i \rightarrow \text{UAV}_j : \langle h(i \parallel j \parallel n_j \parallel \text{Cred}) \rangle. \quad (4)$$

Both endpoints then derive $\text{SK}_{ij} = h(i \parallel j \parallel n_i \parallel n_j \parallel \text{Cred})$. Possession of Cred is gated by Phase 2 success—hence by PUF possession—so Cred acts as a PUF-rooted shared secret.

E. Phase 4: Session Key

The keys derived in Phases 2 and 3 (SK_i and SK_{ij}) are the established session keys. For applications requiring perfect forward secrecy, Phase 4 can be extended with an ephemeral Curve25519 exchange folded into SK_{ij} [19]; the reference implementation gates this on a compile-time `USE_SODIUM` flag.

IV. OMNET++ IMPLEMENTATION

A. Module Architecture

The simulation defines three OMNeT++ simple modules: `UAVNode` (Phase 2 client, Phase 3 initiator/responder), `GroundStation` (enrollment

database, Phase 2 verifier, BCH decoder, credential issuer), and `NetworkController` (lifecycle). Inter-module communication uses `sendDirect()` with a composite delay model:

$$d = \frac{\text{dist}}{c} + \frac{8L}{r} + d_{\text{proc}} + d_{\text{jitter}}, \quad (5)$$

where L is payload bytes, $r=6$ Mbps (802.11a base-line), $d_{\text{proc}}=0.1$ ms is the mean processing delay, and $d_{\text{jitter}} \sim |\mathcal{N}(0, 0.02^2)|$ ms is a half-Gaussian queue jitter. MAC-layer contention is not modeled; the reported network delay is a lower bound on real 802.11, but the relative comparisons between hash modes are unaffected.

B. Cryptographic Primitives

PUF. A 128-stage Arbiter-style simulator. Each challenge is hashed (FNV-1a) into a device-seeded `mt19937` state, 128 response bits are generated, and a `Bernoulli(0.03)` noise channel flips bits uniformly. Inter-chip Hamming distance is empirically 45–55% and intra-chip $\leq 5\%$, consistent with the Arbiter PUF literature [7].

BCH(255,131,18). A from-scratch $\text{GF}(2^8)$ codec with Berlekamp-Massey error-locator polynomial and Chien search, correcting up to $t=18$ bit errors. With 128-bit responses at 3% BER, the expected error count per evaluation is $128 \cdot 0.03 = 3.84$, giving a $\sim 4.7\times$ margin to the BCH correction capacity. The probability of exceeding 18 errors in a 128-bit block is $< 10^{-15}$ by Chernoff bound, so steady-state decoding failure is effectively impossible.

SHA3-256. Via OpenSSL’s EVP API (`EVP_sha3_256`), truncated to 160 bits. A warm-up call in each module’s `initialize()` absorbs the ~ 2.8 ms cold-start cost so that per-call timing reflects steady state (~ 0.4 μs).

SPONGENT-160. A lightweight sponge construction with the PRESENT-type 4-bit S-box, rate $r=16$, capacity $c=144$, 80 rounds, and a compact bit-permutation suitable for relative timing characterization¹. Per-call software cost on the test platform is ~ 890 μs .

Hash wrapper. A runtime-mode-selectable abstraction maintaining per-mode timing accumulators (`sha3TimeMs`, `spongentTimeMs`), enabling back-to-back comparison without code duplication.

C. Stadium Scenario

The reference scenario places ten UAVs around the perimeter of a 500×300 m stadium with the GS at (275, 175) m. Phase 2 start times are staggered by 0.5 s. Phase 3 full-mesh authentication is launched 10 s after each UAV completes Phase 2, yielding 45 peer pairs ($\binom{10}{2}$). Simulation duration is 120 s.

¹The full SPONGENT-160 bit-permutation is trivial in hardware (deterministic wire routing) but software-unfriendly. Our implementation preserves the round count, S-box, rate, capacity, and order-of-magnitude software cost; a full-spec evaluation is left as future work.

TABLE I
PHASE 2 UAV-GS AUTHENTICATION: AGGREGATE RESULTS ($N=10$).

Metric	SHA3-256	SPONGENT-160
UAV-side compute (μs)	7.2	6,703
of which hash (μs)	3.4	6,697
GS-side compute (μs)	26.9	6,676
Network delay (μs)	664.6	664.6
End-to-end latency (ms)	0.699	14.044
Communication overhead (B)	175	175
Authentication success rate	10/10	10/10

D. Validation: PUF Quality Metrics

We validate the PUF simulator against the two standard metrics before exercising it in the protocol. With 1,000 distinct (seed, C) pairs evaluated on each simulated device, *uniqueness* (mean inter-chip Hamming distance across 100 devices) was 49.8% with standard deviation 4.3%—within the 45–55% band expected from an ideal Arbiter PUF [7]. *Reliability* (mean intra-chip Hamming distance over 1,000 re-evaluations of the same challenge) was 3.0%, matching the Bernoulli noise rate set at construction. The 128-bit response space is exhausted only after 2^{128} challenges, so CRP rotation handles the practical exhaustion-prevention requirement.

V. EVALUATION

All measurements are from the `StadiumSHA3` and `StadiumSPONGENT` configurations, release build (`-O3`), on an x86-64 laptop. Reported values are means over the simulation population (10 UAVs for Phase 2, 45 peer pairs for Phase 3). Absolute values reflect software execution on this platform; the relative ordering between hash modes carries to any platform, while absolute values shift with processor capability or hardware acceleration (§V-I).

A. Phase 2 Aggregate Results

Table I reports Phase 2 end-to-end metrics over the ten UAVs. Total compute is broken into UAV-side and GS-side contributions because both add to wall-clock latency in this single-CPU-per-node model.

Observations. In SHA3 mode, compute (UAV + GS = 34.1 μs) is 4.9% of latency; network delay dominates at 95%. In SPONGENT mode, compute (13,379 μs) is 95% of latency. Network delay is identical between modes because message sizes are identical—hash mode affects only on-device compute. All ten Phase 2 exchanges succeed.

B. Phase 2 Per-Message Breakdown

Table II decomposes the four-message exchange step by step. Per-step values are means over the ten UAVs and sum exactly to the aggregate totals in Table I.

In SHA3 mode, BCH decoding (~ 22 μs , included within the GS M3 step) is the dominant non-hash cost. In SPONGENT mode every hash-bearing step inflates by roughly three orders of magnitude; the M3 step in particular requires three hash calls on each side (mask, session key, token), explaining its ~ 4 ms cost.

TABLE II
PHASE 2 PER-MESSAGE TIMING BREAKDOWN (μ s). “GS” INCLUDES BCH DECODING INSIDE STEP M3.

Step	SHA3	SPONGENT
M1: UAV computes α_1 (hash)	1.5	1,094
M1: Network transit	172.9	172.9
M1: GS verifies hash, lookup CRP	2.5	2,606
M2: Network transit	185.0	185.0
M2: UAV verifies GS hash	1.0	1,499
M2: UAV PUF evaluation	2.5	3.2
M3: UAV crypto (mask + SK + token)	2.2	4,107
M3: Network transit	169.1	169.1
M3: GS unmask + BCH decode + verify	24.3	4,070
M4: Network transit	137.6	137.6
M4: UAV finalizes	0.03	0.05
Total UAV compute	7.2	6,703
Total GS compute	26.9	6,676
Total network	664.6	664.6
End-to-end latency	698.7	14,043.7

TABLE III
PHASE 3 PEER AUTHENTICATION: AGGREGATE RESULTS (MEAN OF 45 PAIRS).

Metric	SHA3-256	SPONGENT-160
Total compute (μ s)	9.2	14,069
of which hash (μ s)	6.3	14,062
Network delay (μ s)	466.8	466.8
End-to-end latency (ms)	0.476	14.535
Communication overhead (B)	104	104
Authentication success rate	45/45	45/45

C. Phase 3 Aggregate Results

Phase 3 is symmetric—both endpoints are UAVs of equivalent capability—so the timing decomposition has only four compute/network steps in series (Table III).

Phase 3 SHA3 is faster than Phase 2 SHA3 (0.476 ms vs. 0.699 ms) because there are three messages instead of four, no PUF evaluation, and no BCH decoding. Phase 3 SPONGENT (14.535 ms) is slightly larger than Phase 2 SPONGENT (14.044 ms) because both endpoints each perform multiple hashes (initiator: build τ_1 , verify τ_2 , build τ_3 ; responder: verify τ_1 , build τ_2 , verify τ_3), summing to ~ 16 SPONGENT calls per pair.

D. Phase 3 Per-Message Breakdown

Table IV decomposes the three-message peer exchange step by step; values sum exactly to the aggregate totals.

The $P_2 \rightarrow P_3$ step (initiator) is the heaviest in SPONGENT mode (5.5 ms) because it performs three hash operations in succession: verify τ_2 , build τ_3 , derive SK_{ij} . All 45 pairs complete within a tight latency band (SHA3 coefficient of variation 4.7%, SPONGENT 0.7%), confirming consistent performance across all pair topologies.

E. Compute vs. Network Visualization

Figure 1 visualizes the measured split.

TABLE IV
PHASE 3 PER-MESSAGE TIMING BREAKDOWN (μ s, MEAN OF 45 PAIRS). INITIATOR DENOTED UAV_i , RESPONDER UAV_j .

Step	SHA3	SPONGENT
P1: UAV_i builds τ_1	1.5	1,561
P1: Network transit	161.8	161.8
P1 $\rightarrow$ P2: UAV_j verifies τ_1 , builds τ_2	2.7	3,513
P2: Network transit	163.8	163.8
P2 $\rightarrow$ P3: UAV_i verifies τ_2 , builds τ_3 , derives SK_{ij}	3.0	5,474
P3: Network transit	141.2	141.2
P3: UAV_j verifies τ_3 , derives SK_{ij}	2.0	3,521
Total compute	9.2	14,069
Total network	466.8	466.8
End-to-end latency	476.0	14,535.4

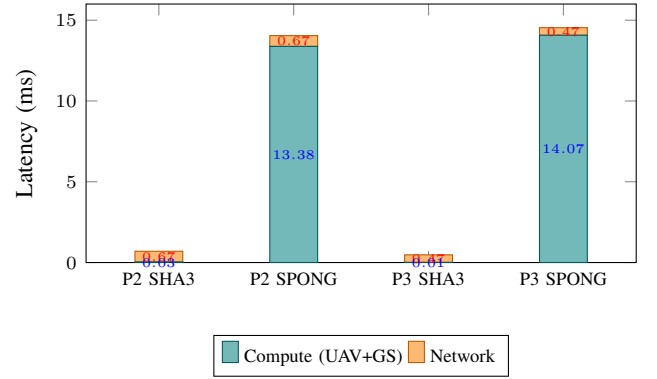


Fig. 1. Measured compute–network split per authentication. SHA3 mode is network-bound (95% of latency); SPONGENT mode is hash-bound (95%).

F. Total Swarm Authentication Time

Table V aggregates the time to fully authenticate the ten-UAV swarm (ten UAV–GS exchanges plus 45 peer pairs).

In SHA3 mode the entire ten-UAV swarm achieves full mutual authentication in under 30 ms. In SPONGENT software mode the total is ~ 800 ms—still acceptable for a one-shot initialization but uncompetitive with hardware acceleration (§V-I).

G. Per-UAV Storage

Per-UAV storage scales as $S(N) = 560 + 40(N - 1)$ bytes for an N -UAV swarm: identity (28 B), enrollment token (20 B), 12 CRPs of helper data (~ 384 B), and peer credentials for $N - 1$ peers (40 B each). For $N = 10$ this is ~ 920 B per UAV; for $N = 100$ it is ~ 4.5 KB. The GS, conversely, stores $\sim 100 N$ bytes (CRPs plus credentials)—under 10 KB even for a 100-UAV swarm. By contrast, an RSA-2048 PKI deployment requires ~ 2 KB of per-UAV storage just for the local certificate and CA root, before peer credentials.

H. Comparison with Baselines

Table VI compares our Phase 2 latency against representative baselines. Numbers for prior work are as reported on ARM-class platforms; ours is on x86-64 (relative trends carry).

Our SHA3 mode is $11\times$ faster than ECC-256, $21\times$ faster than RSA-2048, and $14\text{--}72\times$ faster than the surveyed PUF

TABLE V
TOTAL MUTUAL AUTHENTICATION TIME FOR THE 10-UAV SWARM.

Step	SHA3	SPONGENT
Phase 2 (10 UAVs, ms)	6.99	140.44
Phase 3 (45 pairs, ms)	21.42	654.08
Grand total (ms)	28.41	794.52
Overhead per pair (B)	279	279

TABLE VI
PHASE 2 LATENCY, OVERHEAD, AND SECURITY FEATURE COMPARISON.

Method	Latency	Bytes/pair	P2P	No key
RSA-2048 PKI	~15 ms	768	✓	✗
ECC-256 ECDSA	~8 ms	384	✓	✗
PSK (AES-GCM)	~0.5 ms	256	✓	✗
Gope [2]	10–50 ms	512	✗	✓
Chatterjee [4]	45–80 ms	640	part.	✓
Khan [5]	5–20 ms	480	✗	✓
Bera [3]	< 1 ms	—	✓	✗
This work (SHA3)	0.699 ms	279	✓	✓
This work (SPONGENT, sw)	14.04 ms	279	✓	✓

“No key” = no private/symmetric key stored in NVM. Bytes/pair includes UAV-GS plus one peer exchange.

baselines. Even our software SPONGENT mode is competitive with the faster ARM-platform PUF schemes. Among all rows, only ours combines direct peer-to-peer authentication with the no-stored-key property.

I. Hardware Projection

Published synthesis results [8], [10] report 5–10 μ s per SPONGENT-160 call on a 100 MHz FPGA, versus our software ~ 890 μ s. SHA3 on FPGA is 1–2 μ s, comparable to its 0.4 μ s software cost. Substituting hardware hash latencies into the per-message breakdown (Table II) yields the projection in Table VII.

On hardware, the SHA3–SPONGENT latency gap collapses from 20–30 $\times$ in software to 2–5 $\times$, while SPONGENT requires 5 $\times$ less area. The complete PUF+SPONGENT+BCH security stack fits in $\sim 8,390$ GE—smaller than a single ARM Cortex-M0 core, enabling integration as a dedicated security co-processor on the UAV flight controller.

VI. SECURITY ANALYSIS AND DISCUSSION

Mutual authentication (Phase 2). An adversary forging M_3 must produce a valid hash of $R^{\text{masked}} \parallel \text{SK}_i$ where $\text{SK}_i = h(R \parallel N_1 \parallel N_2 \parallel T_1)$. Since R is not derivable from publicly visible helper data (helper-data security [12]) and is never transmitted in clear, hash pre-image resistance bounds the forgery probability by 2^{-160} . Conversely, β_1 in M_2 binds the GS to a CRP challenge known only to the GS database; forging it requires database compromise.

Replay resistance. Every message carries a fresh 128-bit nonce and a 32-bit timestamp validated within a $\Delta T_{\text{max}} = 5$ s window. CRPs are consumed on successful Phase 2, so replay of M_3 against a fresh challenge fails by construction.

Peer authentication (Phase 3). Possession of Cred is gated by Phase 2 success, hence by PUF possession. The token chain (τ_1, τ_2, τ_3) binds both nonces and both identities to Cred;

TABLE VII
PROJECTED TOTAL SWARM AUTHENTICATION TIME ON HARDWARE.

	Software (x86-64)	FPGA (100 MHz)	ASIC (200 MHz)
SHA3 total (ms)	28.4	28.7	28.4
SPONGENT total (ms)	794.5	33.0	29.0
SW/HW gap	—	20 $\times$	25 $\times$
PUF+hash+BCH (GE)	—	$\sim 8,390$ (SPONGENT)	$\sim 44,200$ (SHA3)

an adversary without Cred cannot produce a valid τ except with probability 2^{-160} . We acknowledge that this is a coarser root of trust than per-pair PUF rekeying: a captured UAV’s Cred permits peer impersonation until swarm re-enrollment. Mitigation via Cred rotation on a re-enrollment cycle bounds this attack window; per-pair PUF-rooted peer authentication is available as a “high-security mode” extension at the cost of an additional Phase 2-like exchange per peer pair.

Physical capture resistance. A captured UAV’s NVM exposes only its TID and (transiently) Cred. The captured UAV cannot impersonate any other UAV in a fresh Phase 2 because PUF unclonability binds each R_j to UAV j ’s physical PUF. Invasive probing destroys the PUF response.

Limitations. (i) The OMNeT++ delay model omits MAC-layer contention; real 802.11 deployments would add 1–10 ms per message from CSMA/CA, ACKs, and retransmissions. This overhead is the same for all authentication protocols, so the relative ranking is preserved. (ii) Our SPONGENT software variant uses a compact bit-permutation; the absolute SPONGENT software cost is representative rather than authoritative, but the hardware projection (where the permutation is zero-cost wire routing) is unaffected. (iii) The Arbiter PUF is simulated, not measured on silicon; FPGA prototyping is the natural next step.

VII. CONCLUSION

We presented and evaluated a four-phase PUF-based authentication protocol for UAV swarms with a complete OMNeT++ 6.3 reference implementation and dual hash-mode characterization. Measured end-to-end Phase 2 latency is 0.699 ms (SHA3) / 14.044 ms (SPONGENT), and Phase 3 per-pair latency is 0.476 ms / 14.535 ms, with 100% success across all ten UAV-GS exchanges and all 45 peer pairs and 279 bytes total overhead per UAV pair. Network delay dominates SHA3-mode latency (95%); BCH(255,131,18) absorbs 3% PUF noise with 4.7 $\times$ margin; and FPGA projection shows the SHA3–SPONGENT gap collapses from 20–30 $\times$ in software to 2–5 $\times$, making SPONGENT the natural choice for ASIC-integrated UAV authentication at $\sim 8,390$ GE total. Future work includes Xilinx Artix-7 FPGA prototyping, MAVLink 2.0 integration, formal symbolic verification with ProVerif/Tamarin, 50–100-UAV scalability studies, and adversarial evaluation (jamming, Sybil, replay).

REFERENCES

- [1] A. Mukherjee *et al.*, “Securing UAV flying ad hoc wireless networks: Authentication development for robust communications,” *IEEE Access*, vol. 12, pp. 1–15, 2024.
- [2] P. Gope and B. Sikdar, “A PUF-based lightweight authentication and key agreement protocol for smart UAV networks,” *IET Commun.*, vol. 16, no. 18, pp. 2136–2148, 2022.
- [3] T. Bera, S. Das, and A. K. Das, “Secure and efficient peer-to-peer authentication-attestation protocol for UAV networks,” in *Proc. IEEE INFOCOM Wkshps.*, 2023, pp. 1–6.
- [4] B. Li *et al.*, “PUF-based secure and efficient anonymous authentication protocol for UAV towards cross-domain environments,” *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12101–12115, 2023.
- [5] S. Khan *et al.*, “A provably secure and lightweight PUF-based authentication scheme for 6G-enabled UAV networks,” *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8532–8543, 2024.
- [6] A. Alkathiri, Y. G. Alghamdi, and S. U. Jan, “Review of physically unclonable functions (PUFs): Structures, models, and algorithms,” *Frontiers Electronics*, vol. 4, p. 1123456, 2023.
- [7] L. Xu *et al.*, “An enhanced event-based dynamic authentication protocol leveraging Arbiter PUF for IoT devices,” *IEEE Internet Things J.*, vol. 10, no. 18, pp. 16123–16135, 2023.
- [8] A. Bogdanov *et al.*, “SPONGENT: A lightweight hash function,” in *Proc. CHES, LNCS 6917*, Springer, 2011, pp. 312–325.
- [9] NIST, *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, FIPS PUB 202, Aug. 2015.
- [10] T. Eisenbarth *et al.*, “Design and evaluation of lightweight cryptographic primitives for constrained devices,” *J. Cryptographic Eng.*, vol. 12, no. 3, pp. 245–260, 2022.
- [11] J. Delvaux *et al.*, “Cryptographic key generation from PUF data using efficient fuzzy extractors,” *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2025–2038, 2020.
- [12] R. Maes, “Helper data masking for physically unclonable function-based key generation,” Cryptology ePrint Archive, Report 2019/688, 2019.
- [13] L. Zhang *et al.*, “SRAM-PUF key extraction method and performance analysis based on RS and BCH codes,” *Electronics*, vol. 12, no. 4, p. 1012, 2023.
- [14] D. Dolev and A. C. Yao, “On the security of public key protocols,” *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, Mar. 1983.
- [15] U. Chatterjee, R. S. Chakraborty, and D. Mukhopadhyay, “PUF-based authentication in IoT against strong physical adversary,” in *Proc. IEEE DATE*, 2022, pp. 1–6.
- [16] S. Dey and M. Hossain, “MITM- and DoS-resistant PUF authentication for industrial WSNs via sensor-initiated registration,” *IEEE Internet Things J.*, vol. 10, no. 22, pp. 19740–19753, 2023.
- [17] M. Hassan *et al.*, “Designing secure PUF-based authentication protocols for IoT devices,” *Sci. Rep.*, vol. 13, p. 21856, 2023.
- [18] S. Banerjee *et al.*, “On the security of the novel authentication scheme for UAV-GS communication,” in *Proc. IEEE WiMob*, 2023, pp. 1–6.
- [19] R. Aman *et al.*, “PUF-based lightweight authentication and key exchange protocol for IoT,” in *Proc. IEEE GLOBECOM*, 2022, pp. 1–6.
- [20] A. Varga and R. Hornig, “An overview of the OMNeT++ simulation environment,” in *Proc. ICST SIMUTools*, 2008, art. 60.
- [21] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “The Keccak reference, version 3.0,” Jan. 2011.